HAFTA 0

Python'a Giriş

Bu dönem boyunca gerçek iş problemleri için veri yapıları ve algoritmalar geliştirmede kullanacağımız programlama aracı.

Bugün Neler Göreceğiz

1. **Veri Yapıları ve Algoritmalar için neden Python**
   Okunabilirlik, ekosistem ve iş dünyasıyla ilgisi
2. **Python'u kurma ve çalıştırma**
   Yorumlayıcı, Jupyter not defterleri, ilk betiğinizi çalıştırma
3. **Değişkenler, veri tipleri ve operatörler**
   Her programın temel yapı taşları
4. **Akış kontrolü: koşullar ve döngüler**
   Karar verme ve tekrar eden işler
5. **Temel veri kapları: liste, demet, sözlük, küme**
   Python'ın hazır veri yapıları
6. **Fonksiyonlar**
   Mantığı yeniden kullanılabilir, test edilebilir birimlere ayırma
7. **Kodlama stili ve iyi uygulamalar**
   Ekip arkadaşlarınızın okuyabileceği kod yazmak

01 / 06

Neden Python?

MOTIVASYON

Bu Derste Neden Python Kullanıyoruz

- Sade İngilizceye yakın, okunabilir sözdizimi — dili öğrenmekle değil, algoritmik düşünmeyle uğraşırsınız
- YBS, analitik ve iş zekasında yaygın kullanılan geniş kütüphane ekosistemi (pandas, NumPy)
- Otomasyon, veri analizi, web arka uçları ve yapay zekâ/ML hatlarında sektör genelinde kullanılıyor
- Jupyter not defterleri gibi etkileşimli araçlar, adım adım deneme yapmayı sağlar — veri yapıları öğrenmek için ideal
- Dinamik tipleme ve hazır yüksek seviyeli yapılar (liste, sözlük), düşük seviye ayrıntılara dalmadan önce kavramlara odaklanmamızı sağlar

YBS Bağlantısı

Bu dönem öğreneceğiniz her veri yapısı — liste, yığın, kuyruk, ağaç — gerçek bilgi sistemlerinde doğrudan bir karşılığa sahiptir: sipariş kuyrukları, organizasyon şeması ağaçları, müşteri veritabanları.

BAĞLAM

Python ile Tipik Bir Alternatifin Karşılaştırması

Python

- Kısa, okunabilir sözdizimi
- Yorumlanır — hızlı geri bildirim döngüsü
- Zengin veri bilimi ve iş kütüphaneleri
- Algoritmaları hızlıca prototiplemek için harika

Java / C++

- Daha ayrıntılı, açık tip belirtimi
- Derlenir — ekstra derleme adımı
- Büyük kurumsal arka uçlarda yaygın
- Bazı ham performans ihtiyaçları için daha iyi

ORTAM

Python'u Kurma ve Çalıştırma

- Python'ı kurun (python.org) veya veri bilimi kütüphaneleriyle birlikte gelen Anaconda dağıtımını kullanın
- Jupyter Notebook / JupyterLab: kodu hücrelerde çalıştırıp sonucu anında gördüğünüz etkileşimli ortam — bu dönemki tüm uygulamalarda kullanılacak
- Alternatif editörler: VS Code, PyCharm — program büyüdükçe faydalı olur
- Python'ı terminalden de çalıştırabilirsiniz: python script.py

UYGULAMA

İlk Python Programınız

```
# yorumlar # ile başlar — Python bu satırı yok sayar
print("Merhaba, YBS Veri Yapıları!")

# print() ekrana çıktı gösterir
print("Ders:", "Veri Yapıları ve Algoritmalar")
```

Çıktı

```
Merhaba, YBS Veri Yapıları!
Ders: Veri Yapıları ve Algoritmalar
```

Bu dönem her Jupyter hücresinde çalıştıracağınız şey aynı basit fikre dayanır: kod yaz, çalıştır, çıktıyı oku.

02 / 06

Değişkenler, Tipler ve Operatörler

TEMEL KAVRAM

Değişkenler

Değişken

Bellekte saklanan bir değere işaret eden isimlendirilmiş bir etikettir. Python'da tip bildirmezsiniz — isim, atadığınız değere bağlanır.

Örnek:

```
musteri_adi = "Ayşe Demir"
siparis_tutari = 249.90
vip_mi = True
```

REFERANS

Python'ın Temel Hazır Veri Tipleri

Tip	Örnek	Tipik İş Kullanımı
int	42	Sipariş adedi, çalışan ID'si
float	19.99	Birim fiyat, vergi oranı
str	"FTR-2045"	Fatura numarası, müşteri adı
bool	True / False	Sipariş ödendi mi? Stok mevcut mu?
NoneType	None	Eksik / henüz atanmamış değer

UYGULAMA

Tip Kontrolü ve Dönüşümü

```
fiyat = "19.99"           # bu bir metin (str), sayı değil
print(type(fiyat))         # <class 'str'>

fiyat = float(fiyat)       # str -> float dönüşümü
vergi = fiyat * 0.18
print(type(fiyat), round(fiyat + vergi, 2))
```

Çıktı

```
<class 'str'>
<class 'float'> 23.59
```

type() neyle çalıştığınızı gösterir; dönüştürme fonksiyonları (int(), float(), str()) tipler arasında geçiş yapar.

Operatörler

- Aritmetik: + - * / // (taban bölme) % (kalan) ** (üs)
- Karşılaştırma: == != > < >= <= → her zaman bool üretir
- Mantıksal: and or not → boolean koşulları birleştirir
- Kısayol atamalar: += -= *= /=

Dikkat

= bir değer atar. == iki değeri karşılaştırır. Bunları karıştırmak en yaygın başlangıç hatalarından biridir.

UYGULAMA

Bir İş Hesabında Operatörler

```
adet = 12
birim_fiyat = 45.0
indirim_orani = 0.10 if adet >= 10 else 0.0

ara_toplam = adet * birim_fiyat
toplam = ara_toplam * (1 - indirim_orani)
print("Ara toplam:", ara_toplam, "| İndirim sonrası toplam:", toplam)
```

Çıktı

```
Ara toplam: 540.0 | İndirim sonrası toplam: 486.0
```

Toplu sipariş indirim kuralı, aritmetik ve karşılaştırma operatörleriyle ifade edildi.

03 / 06

Akış Kontrolü

TEMEL KAVRAM

Koşullar

if / elif / else

Bir programın, bir koşulun True ya da False olmasına göre hangi kod bloğunu çalıştıracağını seçmesini sağlar.

Örnek

```
if stok_adedi == 0:
    durum = "Stokta yok"
elif stok_adedi < 10:
    durum = "Az stok"
else:
    durum = "Stokta var"
```

İŞ SENARYOSU

Müşterinin İndirim Uygunluğuna Karar Vermek

Bir e-ticaret sistemi, her sipariş için müşterinin üyelik seviyesine ve sipariş büyüklüğüne göre sadakat indirimine hak kazanıp kazanmadığına karar vermeli.

- Gold üyeler her siparişte %15 indirim alır
- Silver üyeler 500 TL üzeri siparişte %10 indirim alır
- Diğer herkes tam fiyat öder
- Bu karar mantığı, if / elif / else zincirlerinin doğrudan gerçek dünya kullanımıdır

UYGULAMA

İndirim Kuralını Uygulamak

```
def indirim_orani(seviye, siparis_tutari):
    if seviye == "gold":
        return 0.15
    elif seviye == "silver" and siparis_tutari > 500:
        return 0.10
    else:
        return 0.0

print(indirim_orani("silver", 620))
print(indirim_orani("bronze", 620))
```

Çıktı

```
0.1
0.0
```

TEMEL KAVRAM

Döngüler

for döngüsü

Bir kod bloğunu, bir dizideki (liste, metin, sayı aralığı) her öğe için bir kez tekrarlar.

Örnek

```
for siparis_no in [101, 102, 103]:
    print("Sipariş işleniyor:", siparis_no)
```

UYGULAMA

for ile while Karşılaştırması

```
# for: bilinen bir koleksiyonu dolaş
for isim in ["Ali", "Zeynep", "Mert"]:
    print("Merhaba,", isim)

# while: bir koşul False olana kadar tekrarla
deneme = 0
while deneme < 3:
    print("Giriş denemesi", deneme + 1)
    deneme += 1
```

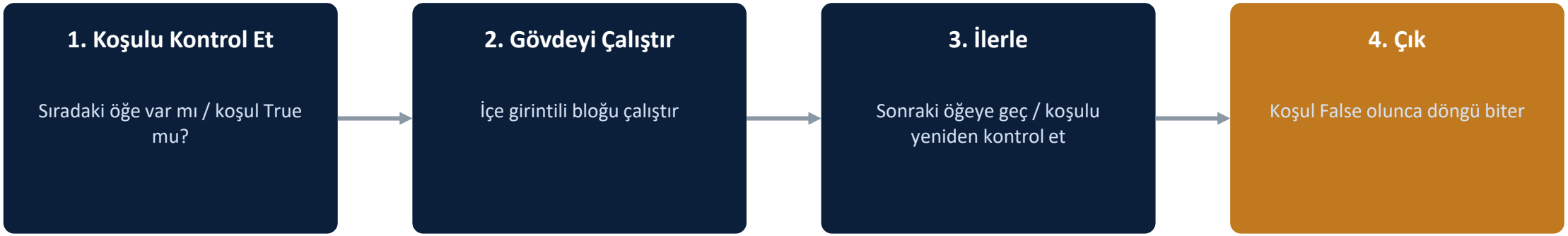
Çıktı

```
Merhaba, Ali
Merhaba, Zeynep
Merhaba, Mert
Giriş denemesi 1
Giriş denemesi 2
Giriş denemesi 3
```

Ne üzerinde döneceğinizi biliyorsanız for kullanın; bir koşul değişene kadar tekrar ediyorsanız while kullanın.

DIYAGRAM

Bir Döngü Nasıl Çalışır



İŞ SENARYOSU

Bir Grup Müşteri Siparişini Toplamak

Bir YBS raporlama işi, bugün verilen her siparişin değerini toplamalı ve risk eşiğinin üzerindeki siparişleri manuel incelemeye işaretlemeli.

- Günün sipariş listesindeki her sipariş üzerinde bir kez döngü kur
- İlerledikçe biriken bir toplamı güncelle
- Döngü içinde bir if ile yüksek değerli siparişleri işaretle
- Bu kalıp — döngü + biriktirici + koşul — iş raporlama kodunda sürekli karşınıza çıkar

04 / 06

Temel Veri Kapları

TEMEL KAVRAM

Listeler

list (liste)

Köşeli parantezlerle yazılan, sıralı ve değiştirilebilir bir koleksiyondur. Oluşturulduktan sonra öğe eklenebilir, çıkarılabilir veya değiştirilebilir.

Örnek

```
urunler = ["Laptop", "Fare", "Monitör"]
urunler.append("Klavye")
print(urunler[0], len(urunler))
```

UYGULAMA

Listelerle Çalışmak

```
envanter = [120, 45, 0, 300]

envanter.append(75)          # sona ekle
envanter[2] = 10             # indeksle bir öğeyi güncelle
az_stok = envanter[1:3]      # dilimleme: 1 ve 2. indeksteki öğeler

print(envanter)
print("Az stok dilimi:", az_stok)
```

Çıktı

```
[120, 45, 10, 300, 75]
Az stok dilimi: [45, 10]
```

TEMEL KAVRAM

Demetler (Tuple) ve Kümeler (Set)

Demet — değişmez ve sıralı

- Parantezle yazılır: (x, y)
- Oluşturulduktan sonra değiştirilemez
- Sabit kayıtlar için uygundur, örn. koordinatlar veya (gün, ay, yıl)

Küme — sırasız, benzersiz öğeler

- Süslü parantezle yazılır: {a, b, c}
- Yinelenenleri otomatik olarak kaldırır
- Hızlı üyelik kontrolü: ogeler in kume

TEMEL KAVRAM

Sözlükler

dict (sözlük)

Anahtar-değer çiftlerinden oluşan sırasız bir koleksiyondur. Bir değeri sayısal konumla değil, anahtarıyla ararsınız — Hafta 10'da işleyeceğimiz hash tablolarının temelidir.

Örnek

```
musteri = {"id": 501, "ad": "Mert Kaya", "vip": True}
print(musteri["ad"])
musteri["email"] = "mert@example.com"
```

KARŞILAŞTIRMA

Doğru Veri Kabını Seçmek

Kap	Sıralı mı?	Değiştirilebilir mi?	Tekrar var mı?	Erişim
list	Evet	Evet	Evet	İndeks (0, 1, 2, …)
tuple	Evet	Hayır	Evet	İndeks
dict	Ekleme sırası	Evet	Anahtar benzersiz	Anahtar
set	Hayır	Evet	Hayır	Üyelik (in)

İŞ SENARYOSU

Bir Müşteri Kaydını Modellemek

Bir CRM, her müşteri profilini öyle saklamalı ki herhangi bir alan, sayısal bir konumu hatırlamaya gerek kalmadan isimle anında bulunabilsin.

- Bir sözlük, her özellik adını (anahtar) değerine eşler: {'ad': ..., 'email': ..., 'seviye': ...}
- Bu tür sözlüklerden oluşan bir liste tüm müşteri tablosunu temsil eder
- Bu yapı, tam olarak bir veritabanı tablosundaki bir satırın veya bir JSON API yanıtındaki bir kaydın şeklidir

05 / 06

Fonksiyonlar

TEMEL KAVRAM

Fonksiyonlar

def

Girdi alan (parametre), bir işi yapan ve isteğe bağlı olarak bir sonuç döndüren, isimlendirilmiş, yeniden kullanılabilir bir kod bloğudur. Fonksiyonlar mantığı bir kez yazıp her yerde yeniden kullanmanızı sağlar.

Örnek

```
def vergi_hesapla(tutar, oran=0.18):
    return tutar * oran

print(vergi_hesapla(200))
```

TEMEL KAVRAM

Parametreler, Dönüş Değerleri ve Kapsam

- Parametreler bir fonksiyonun beklediği girdilerdir; argümanlar ise gerçekten geçtiğiniz değerlerdir
- return, çağırana bir değer gönderir; return içermeyen bir fonksiyon örtük olarak None döndürür
- Varsayılan parametre değerleri (oran=0.18) argümanları isteğe bağlı hale getirir
- Bir fonksiyon içinde oluşturulan değişkenler yereldir — fonksiyon dışında var olmazlar (kapsam)

UYGULAMA

Yeniden Kullanılabilir Bir İş Fonksiyonu

```
def fatura_toplami(birim_fiyat, adet, vergi_orani=0.18):
    ara_toplam = birim_fiyat * adet
    vergi = ara_toplam * vergi_orani
    return ara_toplam + vergi

print(fatura_toplami(150, 3))
print(fatura_toplami(150, 3, vergi_orani=0.08))
```

Çıktı

```
531.0
486.0
```

Bunu bir fonksiyon olarak bir kez yazmak, sistemin her yerinin faturaları aynı, doğru şekilde hesaplamasını sağlar.

06 / 06

Stil ve Özet

Okunabilir Python Yazmak (PEP 8 Temelleri)

- Değişken ve fonksiyon isimleri için snake_case kullanın: siparis_tutari, vergi_hesapla
- Girinti seviyesi başına 4 boşluk kullanın — Python'da girinti kod bloklarını tanımlar
- Değişkenlere anlamlı isimler verin; kısa döngüler dışında tek harf kullanmaktan kaçının
- Kodun sadece ne yaptığını değil, neden yaptığını açıklayan yorumlar ekleyin
- Fonksiyonları kısa tutun ve tek bir işe odaklayın

BİLGİ KONTROLÜ

Hızlı Kontrol 1/2

Bir müşterinin e-postasını müşteri ID'sine göre anında bulmak için hangi veri kabını kullanırsınız?

A Öğe öğe tarayan bir liste

B Müşteri ID'siyle anahtarlanmış bir sözlük

C Tüm e-postaların bir demeti

D ID numaralarının bir kümesi

Neden: Bir sözlük, her öğeyi taramadan anahtarları (ID) doğrudan değerlere (e-posta) eşler.

BİLGİ KONTROLÜ

Hızlı Kontrol 2/2

x = 5. Bu kod ne yazdırır? if x > 10: print('A') elif x > 3: print('B') else: print('C')

A A

B B

C C

D Hiçbir şey — hata verir

Neden: x (5) 10'dan büyük değil, ama 3'ten büyük, bu yüzden elif dalı çalışır.

Uygulama: Sipariş İşleme Isınma Turu

Hafta 0 not defterinizde şunları yapan Python kodu yazın:

- 5 sipariş tutarından (float) oluşan bir liste saklayın
- Her siparişi biriken toplamla birlikte yazdırmak için bir for döngüsü kullanın
- 1000'in üzerindeki herhangi bir siparişi işaretlemek için döngü içinde bir if kullanın
- İndirimli tutarı döndüren bir indirim_uygula(tutar, seviye) fonksiyonu yazın

HINT

Döngü örneğindeki biriktirici kalıbını yeniden kullanın: bir değişkeni 0'dan başlatın, sonra döngü içinde ona ekleyin.

Hafta 0 Özeti

- ✓ Python'ın okunabilir sözdizimi ve zengin ekosistemi, veri yapıları ve algoritmaları öğrenmek için idealdir
- ✓ Değişkenler, tipler ve operatörler her programın en temel yapı taşlarıdır
- ✓ if/elif/else ve for/while, hangi kodun ne kadar çalışacağını kontrol eder
- ✓ Liste, demet, sözlük ve küme, Python'ın hazır veri yapılarıdır — kuracağımız özel yapıların da bir ön izlemesidir
- ✓ Fonksiyonlar, mantığı temiz, yeniden kullanılabilir ve test edilebilir birimlere paketlememizi sağlar

NEXT WEEK

Hafta 1: Algoritmik Düşünme ve Veri Yapılarına Giriş — kod yazmadan önce problemler üzerine nasıl düşüneceğimiz ve doğru veri yapısının neden her şeyi değiştirdiği.